

Primitive Library Design Rationale

Card Games Rule Induction Project

January 2025

1 Introduction: The Design Problem

1.1 The Task

Our goal is to build a domain-specific language (DSL) for expressing card game rules as predicates of type `Hand` $\rightarrow$ `Bool`, where a `Hand` is a list of `Card` objects. This DSL serves a program induction task: given examples of winning and losing hands, the system must synthesize a program that correctly classifies them.

The design of this primitive library must negotiate between two competing imperatives.

1.2 Imperative I: Domain Generality

The first pull is toward *domain generality*—building a library of primitives so fundamental that they could, in principle, apply to list-processing domains far beyond card games. This approach favors the most abstract, compositional operators possible.

The canonical example is DreamCoder [?], whose base library for list manipulation consists of:

Primitive	Type
<code>cons</code>	$a \rightarrow [a] \rightarrow [a]$
<code>car</code> (head)	$[a] \rightarrow a$
<code>cdr</code> (tail)	$[a] \rightarrow [a]$
<code>empty</code>	$[a]$
<code>empty?</code>	$[a] \rightarrow \text{Bool}$
<code>fold</code>	$(a \rightarrow b \rightarrow b) \rightarrow b \rightarrow [a] \rightarrow b$
<code>map</code>	$(a \rightarrow b) \rightarrow [a] \rightarrow [b]$
<code>if</code>	$\text{Bool} \rightarrow a \rightarrow a \rightarrow a$

From this minimal basis, all other list operations—`filter`, `length`, `reverse`, `any`, `all`, `sum`, `max`, and so on—can be derived by composition. The elegance is undeniable: a handful of primitives generates an entire universe of list computations.

1.3 Imperative II: Cognitive Naturalness

The second pull is toward *cognitive naturalness*—ensuring that the primitives align with how humans actually conceptualize card game rules. This matters for two reasons:

1. **Search tractability.** Our synthesis engine enumerates programs by depth. If a natural rule requires depth 7 in a minimal library but depth 2 with convenience primitives, the search time difference is exponential.
2. **Interpretability and transfer.** We want the induced programs to be legible and to support transfer learning. Programs composed of cognitively natural units are easier to interpret and more likely to share substructure with related rules.

Consider the rule “the sum of ranks exceeds 21” (as in Blackjack). In DreamCoder’s minimal library, this requires something like:

```
gt (fold ( c  acc. + (rank_val c) acc) 0 hand) 21
```

But humans describe this rule as “the total is over 21”—a single conceptual unit. With a dedicated `sum_ranks` primitive, the program becomes:

```
gt (sum_ranks hand) 21
```

The depth reduction from ~ 7 to ~ 2 translates directly to orders-of-magnitude speedups in enumeration-based synthesis.

1.4 The Chicken-and-Egg Problem

These two imperatives create a methodological tension. To know which convenience primitives are worth including, we need some notion of the rules we intend to cover. But part of the point of building this library is to have a principled criterion for selecting rules—specifically, rules where we expect interesting transfer effects and where explicit reasoning can leverage compositional structure.

We address this circularity through an empirical bootstrapping process:

1. **Start with a garden-variety rule catalogue.** We compiled two rule sets: (a) 44 “pre-training” rules inspired by familiar card games (Poker, Blackjack, Rummy, Solitaire), and (b) 45 “experimental” rules spanning a range from simple positional checks to exotic structural patterns (palindromes, bracket matching, arithmetic progressions).
2. **Analyze compositional structure.** For each rule, we wrote out its logical form in terms of candidate primitives, identifying recurring patterns and bottlenecks.
3. **Iterate on the library.** Where a natural rule required excessive composition depth, we considered adding a convenience primitive. Where primitives were rarely used or easily derived, we considered removing them.

The result is a library that balances generality with tractability—not minimal in the theoretical sense, but optimized for the empirical distribution of rules we care about.

2 The List Processing Core

In building this library, we start with the fact that our card hands are list objects. The goal of the task is to find a function of type `Hand → Bool`, where `Hand` is a list. As a result, our language must have list primitives capable of extracting patterns from lists in general.

2.1 The Minimal Basis We Did Not Choose

From a theoretical standpoint, list operations can be reduced to a minimal set of five primitives: `nil` (empty list), `cons` (prepend), `head` (first element), `tail` (rest of list), and `null?` (emptiness test). Combined with recursion, these generate all list operations. More powerfully, a single operation—`fold`—serves as the universal list eliminator, from which `map`, `filter`, `any`, `all`, `length`, `sum`, `max`, and virtually every other list-consuming function can be derived.

We deliberately rejected this minimal approach for the reasons outlined above. Consider concrete examples from our rule catalogue:

Example: Flush (all same suit). The rule `poker_flush` checks whether all cards share a suit. In our library:

```
eq 1 (length (unique (map get_suit hand)))
```

With only `fold`, this becomes:

```
eq 1 (length (fold ( c  acc . if (member (get_suit c) acc)
                        acc
                        (cons (get_suit c) acc))
                  [] hand))
```

The depth explosion is evident.

Example: Sorted by rank. The rule `Sorted_by_rank` (`r1x`) checks non-decreasing order:

```
all ( p . le (rank_val (fst p)) (rank_val (snd p))) (adjacent_pairs hand)
```

Without `adjacent_pairs` as a primitive, constructing pairs requires `zip-with` applied to the list and its tail—adding several layers of composition.

2.2 Our List Primitives

We include the following list operations, organized by function:

2.2.1 Positional Access

Primitive	Type	Purpose and Example Rules
<code>head</code>	$[a] \rightarrow a$	First element. Used in <code>simple_first_red</code> : “the first card is red” $\rightarrow$ <code>eq RED (get_color (head hand))</code>
<code>last</code>	$[a] \rightarrow a$	Final element. Used in <code>simple_last_black</code> : “the last card is black” $\rightarrow$ <code>eq BLACK (get_color (last hand))</code>
<code>at</code>	$\text{Int} \rightarrow [a] \rightarrow a$	Index access. Used in <code>comp_first_two_same_suit</code> : <code>eq (get_suit (at hand 0)) (get_suit (at hand 1))</code>
<code>length</code>	$[a] \rightarrow \text{Int}$	Size. Used in counting rules and as denominator in “majority” predicates.

2.2.2 Slicing and Restructuring

Primitive	Type	Purpose and Example Rules
<code>take</code>	$\text{Int} \rightarrow [a] \rightarrow [a]$	First n elements. Used in half-based rules: <code>take (half_len hand) hand</code>
<code>drop</code>	$\text{Int} \rightarrow [a] \rightarrow [a]$	Skip first n . Used for right-half access and shifted pairs.
<code>reverse</code>	$[a] \rightarrow [a]$	Reverse order. Essential for palindrome rules like <code>Suits_palindrome</code> : <code>eq (map get_suit hand) (reverse (map get_suit hand))</code>
<code>zip_with</code>	$(a \rightarrow b \rightarrow c) \rightarrow [a] \rightarrow [b] \rightarrow [c]$	Parallel combination. Used to construct adjacent pairs and shifted pairs for relational patterns.

2.2.3 Higher-Order Operations

Primitive	Type	Purpose and Example Rules
<code>map</code>	$(a \rightarrow b) \rightarrow [a] \rightarrow [b]$	Transform each element. Ubiquitous: <code>map get_suit hand</code> , <code>map rank_val hand</code> .
<code>filter</code>	$(a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$	Select matching elements. Used in <code>count_two_red</code> : <code>eq 2 (length (filter (c. eq RED (get_color c)) hand))</code>
<code>any</code>	$(a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow \text{Bool}$	Existential quantification. Used in <code>simple_has_spade</code> : <code>any (c. eq SPADES (get_suit c)) hand</code>
<code>all</code>	$(a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow \text{Bool}$	Universal quantification. Used in <code>sol_ascending</code> : <code>all (p. lt (rank_val (fst p)) (rank_val (snd p))) (pairs hand)</code>
<code>unique</code>	$[a] \rightarrow [a]$	Remove duplicates. Essential for counting distinct values: <code>length (unique (map get_suit hand))</code>

2.2.4 Aggregation (In Lieu of Fold)

Primitive	Type	Purpose and Example Rules
<code>sum_ranks</code>	$\text{Hand} \rightarrow \text{Int}$	Total of rank values. Would be <code>fold (+) 0 (map rank_val hand)</code> . Used in <code>bj_sum_even</code> : <code>eq 0 (mod (sum_ranks hand) 2)</code>
<code>max_rank</code>	$\text{Hand} \rightarrow \text{Int}$	Highest rank. Would require fold with conditional.
<code>min_rank</code>	$\text{Hand} \rightarrow \text{Int}$	Lowest rank. Symmetric to <code>max_rank</code> .

These aggregates would be derivable from `fold`, but we include them as primitives because they

appear frequently in real card games and their explicit inclusion dramatically reduces search depth.

The cost of omitting **fold**: if a rule needs an unusual aggregation (say, “product of ranks mod 7”), we would need to add a new primitive rather than expressing it compositionally. In practice, our rule catalogue does not require such exotic aggregations.

3 Derived Operations Included for Cognitive Naturalness

Beyond the core list machinery, we include several operations that are technically derivable from other primitives but which correspond to natural units of thought in how humans reason about card games.

3.1 Structural Conveniences

Primitive	Derivation	Cognitive Motivation
<code>last</code>	<code>head (reverse xs) or at (length xs - 1) xs</code>	“The last card” is a natural referent; nobody thinks “head of the reversal.” Used in rules like <code>Ends_same_suit</code> (r44x).
<code>first_half</code>	<code>take (half_len xs) xs</code>	Many games involve comparing halves. Used in <code>Halves_copy_suits</code> (r21x): “left suits match right suits.”
<code>second_half</code> <code>half_len</code>	<code>drop (half_len xs) xs</code> <code>div (length xs) 2</code>	Symmetric to <code>first_half</code> . Supporting primitive for half-based patterns.
<code>adjacent_pairs</code>	<code>zip_with pair xs (drop 1 xs)</code>	Rules about consecutive cards are pervasive: <code>sol_alternating</code> (“colors alternate”), <code>Adj_same_rank_or_suit</code> (r38x).

Games frequently reference structural positions directly: “split the deck,” “the last card played,” “each consecutive pair must differ by one.” Including these as primitives respects the grain of human card-game discourse.

3.2 The Color Abstraction

Primitive	Derivation	Cognitive Motivation
<code>get_color</code>	<code>if (or (eq s HEARTS) (eq s DIAMONDS)) RED BLACK</code> <code>where s = get_suit card</code>	Color is phenomenologically primary, not derived.

Technically, color is a function of suit—red for hearts/diamonds, black for clubs/spades. But cognitively, color is often perceived as a primitive attribute. People say “alternate red and black” without decomposing this into suit logic. Rules like `Uniform_color` (r3x: “all cards are the same color”) and `sol_alternating` (“colors alternate throughout”) are stated in terms of color directly.

The same logic motivates the alternative color groupings `get_altcolor1` (Pointy: ♠♦ vs. Round: ♥♣) and `get_altcolor2` (SH: ♠♥ vs. DC: ♦♣) used in rules like `AltColor1_palindrome` (r27x).

3.3 Direct Query Primitives

Primitive	Derivation	Example Rule
<code>has_suit</code>	<code>any (c. eq (get_suit c) s) hand</code>	<code>simple_has_spade</code> : “hand contains at least one spade”
<code>has_color</code>	<code>any (c. eq (get_color c) c) hand</code>	“Is there a red card?”
<code>count_suit</code>	<code>length (filter (c. eq (get_suit c) s) hand)</code>	<code>Exactly_one_club</code> (r43x): “exactly one ♣”
<code>count_color</code>	<code>length (filter ...)</code>	<code>count_two_red</code> : “exactly 2 red cards”
<code>n_unique_suits</code>	<code>length (unique (map get_suit hand))</code>	<code>poker_flush</code> : “all same suit” $\equiv$ unique suits = 1
<code>n_unique_ranks</code>	<code>length (unique (map get_rank hand))</code>	<code>poker_has_pair</code> : unique ranks < length implies a pair
<code>n_unique_colors</code>	<code>length (unique (map get_color hand))</code>	<code>poker_same_color</code> : unique colors = 1

These are all one-liners in terms of more basic operations, but they correspond to questions players naturally ask. “Do I have any spades?” is a single cognitive unit, not a quantified predicate over suit-equality. Including these as primitives compresses common queries into single tokens, both accelerating search and aligning the DSL with human conceptualization.

3.4 Comparison Conveniences

Primitive	Derivation	Cognitive Motivation
<code>all</code>	<code>not o any o (not o p)</code>	“All cards satisfy P” is as natural as “some card satisfies P.” Used pervasively.
<code>le</code>	<code>or (lt x y) (eq x y)</code>	“At most” is common in threshold rules.
<code>gt</code>	<code>lt y x</code>	“Greater than” is symmetric to “less than.”
<code>ge</code>	<code>not (lt x y)</code>	“At least” appears in minimum-threshold rules like <code>count_three_suit</code> .

While `any` and `lt` suffice in principle, the derived comparisons and quantifiers map directly to English phrasing (“at least 3,” “all cards,” “greater than 10”) and are included to preserve this alignment.

4 Card-Specific Primitives

Beyond list operations, we need primitives for accessing card attributes.

4.1 Record Accessors

Primitive	Type	Status
<code>get_suit</code>	$\text{Card} \rightarrow \text{Suit}$	Primitive. Extracts suit ($\clubsuit, \diamondsuit, \heartsuit, \spadesuit$).
<code>get_rank</code>	$\text{Card} \rightarrow \text{Rank}$	Primitive. Extracts rank (2–A).
<code>rank_val</code>	$\text{Card} \rightarrow \text{Int}$	Primitive. Numeric value (2–14).
<code>get_color</code>	$\text{Card} \rightarrow \text{Color}$	Derived (from suit), but included for cognitive naturalness.
<code>get_parity</code>	$\text{Card} \rightarrow \text{Parity}$	Derived (from <code>rank_val</code> mod 2). Used in <code>Uniform_rank_parity</code> (r48x).

4.2 Constants

We include a minimal set of constants:

- **Suits:** CLUBS, DIAMONDS, HEARTS, SPADES
- **Colors:** RED, BLACK
- **Integers:** 0–5 only (for counting; no arbitrary thresholds like 21)
- **Booleans:** true, false

Notably, we exclude rank constants (e.g., 11 for Jack, 14 for Ace). This was a deliberate design decision: rules like “has a face card” or “sum exceeds 21” require arbitrary numeric thresholds that don’t transfer well across rule families. By restricting to small counting constants, we force the library toward structural patterns rather than domain-specific magic numbers.

5 Boolean and Arithmetic Operations

5.1 Boolean Logic

Primitive	Type	Status
<code>and</code>	$\text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$	Primitive. Logical conjunction.
<code>or</code>	$\text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$	Primitive. Logical disjunction.
<code>not</code>	$\text{Bool} \rightarrow \text{Bool}$	Primitive. Logical negation.
<code>if</code>	$\text{Bool} \rightarrow a \rightarrow a \rightarrow a$	Primitive. Conditional.

These form a complete basis for propositional logic.

5.2 Comparisons

Primitive	Type	Status
<code>eq</code>	$a \rightarrow a \rightarrow \text{Bool}$	Primitive. Polymorphic equality.
<code>lt</code>	$\text{Int} \rightarrow \text{Int} \rightarrow \text{Bool}$	Primitive. Strict less-than.
<code>le</code> , <code>gt</code> , <code>ge</code>	$\text{Int} \rightarrow \text{Int} \rightarrow \text{Bool}$	Derived from <code>lt</code> , <code>eq</code> , <code>not</code> . Included for naturalness.

5.3 Arithmetic

Primitive	Type	Status
<code>+</code>	$\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$	Primitive. Addition.
<code>-</code>	$\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$	Primitive. Subtraction.
<code>mod</code>	$\text{Int} \rightarrow \text{Int} \rightarrow \text{Int}$	Primitive. Modulo. Used for parity: <code>eq 0 (mod (sum_ranks hand) 2)</code>

6 Expressiveness and Limitations

6.1 What the DSL Can Express

Our library supports:

- **Regular patterns:** “All hearts,” “alternating colors,” “first card is red.”
- **Bounded counting:** “Exactly 3 hearts,” “more red than black,” “at least half same suit.”
- **Linear scans with bounded memory:** “Sorted by rank,” “all adjacent pairs satisfy R.”
- **Parallel structure comparison:** “Halves have same suit sequence,” “palindromic colors.”
- **Arithmetic progressions:** “Contains a 3-term rank pattern with equal steps.”

6.2 What the DSL Cannot Express

- **Pushdown patterns:** Bracket matching (e.g., `Well_formed_brackets_by_suit`, `r15x`) requires stack semantics. We include this as a specialized primitive rather than deriving it.
- **Unbounded recursion:** No Turing-complete computation.
- **Variable cross-element dependencies:** “Card i relates to card $i + k$ ” for variable k is limited.

6.3 The Fold Trade-off

The absence of `fold` is the most significant expressiveness limitation. With `fold`, arbitrary reductions become expressible:


```

-- Product of ranks
fold (*) 1 (map rank_val hand)

-- Running maximum
fold ( c  acc. if (gt (rank_val c) acc) (rank_val c) acc) 0 hand

-- Custom aggregation
fold ( c  acc. if (condition c) (f acc) acc) init hand

```

Without `fold`, we must pre-specify which aggregations to support (`sum_ranks`, `max_rank`, `min_rank`). This is a deliberate trade-off: we sacrifice expressiveness for search tractability, betting that our pre-specified aggregations cover the empirically relevant rules.

7 Summary: The Final Library

Our library contains 59 base primitives organized as follows:

Category	Count
Constants (suits, colors, integers, booleans)	14
Card accessors	4
Positional access	5
List slicing	7
Direct queries	7
Aggregates	3
Comparisons	5
Boolean operations	4
Higher-order functions	5
Arithmetic	3
Total	59

Of these, roughly 27 are “truly primitive” in the sense of being irreducible, while the remaining 32 are derived but included for cognitive naturalness and search efficiency.

The guiding principle throughout has been: *optimize for the empirical distribution of rules we care about, not for theoretical minimality*. The result is a library that makes natural rules easy to express while remaining general enough to support transfer learning across rule families.

References

- [1] Ellis, K., Wong, C., Nye, M., Sablé-Meyer, M., Morales, L., Hewitt, L., ... & Tenenbaum, J. B. (2021). DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. *PLDI 2021*.